

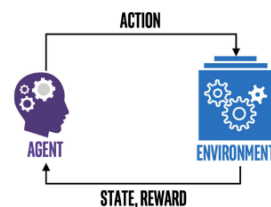
Session 2: Reinforcement Learning 101

🕒 Created	@May 12, 2022 12:18 PM
📁 Class	Reinforcement Learning
📁 Type	
🔗 Materials	
☑ Reviewed	☐
☰ Property	

Basic RL Concepts

Reinforcement Learning Goal

- The **goal of reinforcement learning** is to learn an optimal strategy for achieving a goal, by experimental trials and relatively simple feedback.
- The agent actively adapts to an environment to maximize future rewards.



Finding best strategy for solving a problem- we don't have this concept of accuracy (perfectly) → simplifying and doing trial and error. We need a feedback (we need to see how the actions)

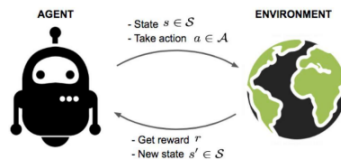
Key Concepts

- An **Agent** (element performing the actions) acts in an unknown **Environment** (where it takes the actions -sometimes you are able to model it with a mathematical function) e.g. chess is concrete and easy to define.
 - Can we model environment or not? **model-based** or **model-free** environment. Other times → hard to define e.g. autonomous driving.
- The **Interaction** with the Environment gives **Reward** to the Agent.
 - (Each action receives a reward)
 - Reward: measures how well agent is doing (it is a number) it depends how we are defining our policy.
- An Agent can stay in one or many **States** of the Environment and choose to take one or many **Actions**.
 - The agent can stay in many states: what is a state? The given state we have in our system.
 - It is a given situation in an environment. Where the agent is positioned in the environment. It defines the state (a given position).
 - It sometimes defines if it is able to do it or not e.g. in chess, you can only take specific moves defined. Some states are allowed and sometimes not.

From moving from one state to the other we have **Transition probabilities** → Problems have randomness (some stochastic property) e.g. we will have problems that we cannot anticipate e.g. in driving, we will not know how likely we are going to find a car driving in the next street.

- Which State the Agent will arrive is decided by **Transition probabilities** between States.
 - Some randomness happening in the system- in the real world there are some problems we cannot anticipate.

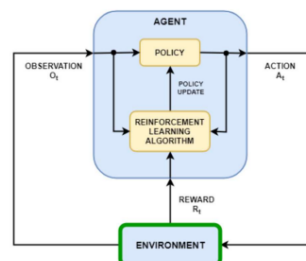
- g. if were driving you cannot know who youre going to find (element of randomness).



A **policy** is like a script → it is a mathematical function, recipe that your agent is going to follow. The goal that we have in this system is to improve this policy → e.g. make perfect cake. We have to make policy better in those iterations (continually learning for adapting policy and making it better) and we do this by maximizing the **value function** which is the way in which we measure the future rewards → how are we going to maximize the rewards that I am going to get in my system. We maximize value function to maximize our rewards. The value function is going to quantify how good is a given state for achieving our goal i.e if i am achieving in this position, how good is this state for achieving the goal i am trying to get → it is going to predict what is the expected amount of reward that I am going to get e.g. if I am playing chess and i am considering if to do this move or the other, this value function is going to tell me what is the move that is going to increase the probability that i am going to win the match.

- The Agent's **Policy** provides a guideline on what is the optimal action to take in a particular State.
- The goal is to Maximize the Total Rewards.
- Each State is associated with a **Value Function**.
- A Value Functions predicts the expected amount for future Rewards we can receive in this State, by following the corresponding Policy.
- A Value Function quantifies how good a state is.
- Both Policies and Value Functions are what we try to learn in RL.

We have to continuously improve, adapt and improve that policy and this is by maximixing the **value function** and which is the way we measure the future reward. It predicts what is the expected amount of future rewards. This value function tells me what is the move that is going to improve the probability that I win my match.



Key Concept Examples

Chess Game Example

We want to win a chess game.

- Environment: chess board
- Agent: player
- Reward: Win or lose the game (this is binary in this case)
- States: each possible board configuration (the position of the pieces)
- Actions: the set of all possible moves from a particular state and for a particular player
- Policy: (how to play) → optimal move from a state
- Value state: how good is the current configuration i have in the game for playing (how good is the current set up)

Problem	Win a Chess game
Environment	Chess Board
Agent	Player
Reward	Win or loose the game
States	Each possible board configuration
Actions	The set of all possible moves from a particular state, and for a particular player
Policy	Optimal move from a state
Value State	How good the board configuration is for the player, where the goal is to win the game

House Cleaning Robot

The robot is cleaning house and it has to find the recharge station because it is running out of battery. The problem is that it doesn't have a model of your house.

- Environment: set of rooms to which the robot has access.
- Agent: robot.
- Reward: finding the recharging station (how am I going to give it a price for getting closer).
- States: the position of the robot with respect to the recharge station ; and also with respect to current charge level.
 - Why add charging? The optimal combination (we get there faster, a way to maximize it).
 - You cannot measure distance because you don't know.
 - You have a limited number of moves and every time you make a move you lose points; way of making him do it faster.
- Actions: the movements (right, left, ahead, back)
- Policy: optimal set of moves that I have for a given state.
- Value state: charged level of the robot, and where the goal is to find recharge.

Problem	Find the recharge station
Environment	The set of rooms to which the robot has access
Agent	Robot
Reward	Find the recharge station
States	Current charge level, position related to the recharge station
Actions	Right, Left, Ahead, Back
Policy	Optimal move from a state
Value State	The current charge level for the robot, where the goal is to find the recharge station

Elements of RL

Policy $\pi(s)$

How do you define policy in Mathematical form. In this situation take this action, in this other situation take this other situation.

- Defines the learning agent's way of behaving at a given time.
- Policy is mapping from perceives states of the environment, to actions to be taken when in those states.
- Follows stimulus-response rules or associations.
- In general, policies may be stochastic, specifying probabilities for each action.

A policy can be deterministic $\rightarrow \pi(s) = a$

- i.e. you know that this is the best action you can take

A policy can be stochastic $\rightarrow \pi(a|s) = \mathbb{P}_{\pi}[A = a|S = s]$

- i.e. how likely it is, that in a given state if you take this policy, how likely it is that you achieve your goal.

Value Function $V(S)$

Tells you in the long term how good it is to be in a state for achieving a goal → how likely it is to follow, find the goal and get rewards you want. In other words, values are predictions of rewards (that is the concept of the value). What are the rewards I am going to get in the future if I take this action in this given state.

- It is the most important component in all algorithms is how to efficiently estimate those values as it is very complex to anticipate what is the reward I am going to get in a given scenario.

How to efficiently estimate those values (can be complex to anticipate what is the reward I will get in a given case scenario).

- Indicates the **long-term** desirability of states, after taking into account the states that are likely to follow and the rewards available in those states
- The value of a state is the **total amount of reward** an agent can expect to accumulate over the future, starting from that state
- In a sense, **values are predictions of rewards**
- Action choices are made based on value judgments
- Values must be estimated and re-estimated from the sequences of observations an agent makes over its entire lifetime
- **The most important component of almost all reinforcement learning algorithms we consider is a method for efficiently estimating values**

Episodes

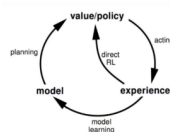
Used for helping us having a beginning and end; so we can define training to our models; update our policies. Episodes are a series of states or actions. Episodes is a sequence of states and actions.

- State action reward → sequence until end of episode.
- Sometimes you cant finish episode e.g. autonomous driving (when you crash on wall)
- The interaction between the agent and the environment involves a sequence of actions and observed rewards in time, $t = 1, 2, \dots, T$
- During the process, the agent accumulates knowledge about the environment and makes decisions on which actions to take next so as to efficiently learn the best policy.
- Let's label the state, action, and reward at time step t as: S_t , A_t and R_t
- Thus the interaction, sequence is fully described by one **episode** (also known as "trial" or "trajectory"), and the sequence ends at the terminal state S_T :

$$S_1, A_1, R_1, S_2, A_2, \dots, S_T$$

Model

- It mimics the behavior of the environment, or more generally, allows inferences to be made about how the environment will behave
- Ex: Given a state and action, the model might predict the resultant next state and next reward
- Models are used for planning, by which we mean any way of deciding on a course of action by considering possible future situations before they are actually experienced
- A RL method can be **model -based** (chess) or **model free** (autonomous driving → take a decision and see how it goes)



If you do not have a model based problem, and learn on the fly by taking actions and decide what is happening; then this is called a model-free. Can either be model based or model free approach.

- Having a model allows you to plan/prepare/to train offline → to make sure what is the best move I am going to make (allows you to train offline)
- When you do not have that model, you do not have that benefit and you have to experiment. In this type of situations you have to go to production (you have to experiment).

If we know the model

- We can plan with perfect information.
- It is called **model-based RL**
- We can find the optimal solution (using Dynamic Programming (DP)) → you can see all the possible variations.
- Rely on the model of the environment; either the model is known or the algorithm learns it explicitly.

If we don't know the model

- We learn from incomplete information.
- It is **model-free RL**
- No dependency on the model during learning

Brief history of Reinforcement learning

1. Psychology of animal learning

- How humans and animals learn (starts at early 1980s) → earliest work in AI.
- The first implementation was seen way back in 1911 by Edward Thorndike defined the "Law of Effect"
 - Trial and error learning, which says *"animals are likely to recur to satisfaction and less likely to discomfort."* → makes sense as we are driven by pleasure.
- English translation of Pavlov's "Classical Conditioning" 1927 → is way we see this effect of law; there is a trial and error and you get a reward by doing that.

How do you translate this into computers?

- Alan Turing in 1948 talked about the "pleasure pain system":
 - When you have a situation in a computer that is equivalent to pleasure; in our case is to get to the goal we are trying to achieve e.g. win chess game, drive successfully our autonomous driving car, all of the other actions are equal to 0. Mathematically speaking, we have a way to define which are the best actions to take.
 - Possible to program a computer following the same idea → *"when a stimulus is equivalent to pleasure, the other tentative entries are cancelled"*
 - Experiment, basically where they are introducing some cages that were having complex systems for opening the cage. The cat was trying different things to escape. Eventually, he was learning how to break system. With time, cats were finding the solution better (better and better with trial and error).
- The first implementation that we can see that is close to RL is the Matchbox Educable Naughts and Crosses Engine (MENACE).
 - The MENACE machine came on the first idea that you can program a system on trial and error learning.
 - Other systems like Tic-tac-toe (Donald Michie 1961 and 1963)
- "N-armed bandit" it's the way Netflix decides what movie to show you → and stochastic learning automata, 1960.
 - Based on a model that is trying things.
- This led to other research on economic, financial applications and a combination with game theory.
- Classifier systems, genetic algorithms (John Holland 1975)
- Separated from supervised learning (more popular), "the hedonistic neuron" research of Harry Klopf 1972 -1982

- Sutton and Barto (fathers of RL; studied from 1981 to 1997)

Menace Machine Video

- Mimic how humans try to learn. If we see something is working, we repeat; if not, we do not.

There is another branch of reinforcement learning that comes from **optimal control**.

2. Optimal control with value functions and dynamic programming, at late 1950s

- When you have sensors, close loop/control loop. Dynamic air conditioning in big data centres like Google.
- Bellman equation (Richard Bellman, 1950 -1957) → *Optimal return function*
- The origin's of this was something called Markov Decision Process (MDP) because it is the fundamental equation of RL → first way of solving RL using computers.
- Problem "The curse of dimensionality" Bellman → *The computational requirements grow exponentially with the number of state variables, still far more efficient and more widely applicable than any other general method.*

3. Temporal Difference Methods

There is a third source of the history of RL, which is the first time they started considering the time E.g. in chess you can take your time for thinking.

- We have to find a way to simulate the time our equation → with a hyperparameter; therefore we are going to be able to tell model, how much to remember about time.

The three origins of RL converge in late 1980s to produce the modern **Reinforcement Learning** we know until now.

4. Modern Reinforcement Learning

- Started with what we call the "Deep Q-Network (DQN)" in 2015
- Other researchers: AlphaGo onwards (David Silver) plays Go, 2015
 - AlphaGo Zero masters Go without human knowledge, 2017
 - AlphaZero extended to chess/shogi, 2018
 - MuZero combining with a learned model, 2019-2020
 - Many more gaming contributions, 2017-2020
 - Many new algorithms and methods, 2010-2020
 - RL applied to:

✓ Healthcare	✓ Computer systems	✓ Business management
✓ Education	✓ Gaming	✓ Science engineering art
✓ Transportation	✓ Robotics	✓ Among others...
✓ Energy	✓ Computer vision	
✓ Finance	✓ NLP	

AlphaZero

- Moment where AlphaGo developed by Deep Mind (part of Google) founded by David Silver.
- Developed system called AlphaGo, bot trained to play a game in Asia which is the chess in Asia which is much more complex.
- Machine was able to learn from all players before and took all of the plays to teach a system to learn and adapt.
 - You took all the plays, isn't it supervised learning? They indeed took those actions for updating the policy and therefore they said; how can we do this even better than humans? And think other options that are not bias. That's why they

created AlphaZero where there is no previous information in system (Alpha Zero → model free and Alpha Go → model based).

- At the beginning machine was very lousy but after a few hours, beat the world champion.

Some Practicle Examples

- “Eat Melon” is basically a simulation environment that is using Deep Q reinforcement learning (Sess 4/5).
- The idea is that this model has an environment and it has to find all the fruits and sometimes gets one of those brown things and we have some obstacles too.
- At the begining its really bad.

Eat Melon!

This demo is a fun way to gain familiarity with the Deep Q Learning algorithm. It is based off Karpathy's ConvNetJS Deep Q Learning Demo with some tweaks on the UI side. You can fiddle with reward and model parameters, see how that affects the agent, which should give you a feel for how Deep Q learning works.

<http://projects.rajivshah.com/rldemo/>

Q-Learning-Demo-Play-nChain/q_learning_notebook.ipynb at master · MauroLuzzatto/Q-Learning-Demo-Play-nChain

This repository contains a Jupyter Notebook with an implemenation of a Q-Learning Agent, which learns to solve the n-Chain OpenAI Gym environment - Q-Learning-Demo-Play-nChain/q_learning_notebook.ipynb at master · MauroLuzzatto/Q-Learning-Demo-Play-nChain

https://github.com/MauroLuzzatto/Q-Learning-Demo-Play-nChain/blob/master/q_learning_notebook.ipynb

MauroLuzzatto/Q-Learning-Demo-Play

This repository contains a Jupyter Notebook implementation of a Q-Learning Agent, which learns to solve the n-Chain OpenAI...

1 Contributor 0 Issues 1 Star

amazon-sagemaker-examples/reinforcement_learning/rl_traveling_salesman_vehicle_routing_coach at main · aws/amazon-sagemaker-examples

The travelling salesman problem (TSP) is a classic algorithmic problem in the field of computer science and operations research. Given a list of cities and the distances between each pair of cities, the problem is to find the shortest possible route that visits each city and returns to the origin city.

https://github.com/aws/amazon-sagemaker-examples/tree/main/reinforcement_learning/rl_traveling_salesman_vehicle_routing_coach

amazon-sagemaker-examples/reinforcement_learning/rl_hvac_coach_energyplus at main · aws/amazon-sagemaker-examples

HVAC stands for Heating, Ventilation and Air Conditioning and is responsible for keeping us warm and comfortable indoors. HVAC takes up a whopping 50% of the energy in a building [1, 2] and accounts for 10% of global electricity use [3], and several control system optimizations have been proposed to reduce this energy use while ensuring thermal comfort [4, 5, 6].

https://github.com/aws/amazon-sagemaker-examples/tree/main/reinforcement_learning/rl_hvac_coach_energyplus

aws/amazon-examples

Example Jupyter notebook how to build, train, and deploy models using Amazon SageMaker

399 Contributors 528 Issues

Q-Learning-Demo-Play-nChain/q_learning_notebook.ipynb at master · MauroLuzzatto/Q-Learning-Demo-Play-nChain

This repository contains a Jupyter Notebook with an implemenation of a Q-Learning Agent, which learns to solve the n-Chain OpenAI Gym environment - Q-Learning-Demo-Play-nChain/q_learning_notebook.ipynb at master · MauroLuzzatto/Q-Learning-Demo-Play-nChain

https://github.com/MauroLuzzatto/Q-Learning-Demo-Play-nChain/blob/master/q_learning_notebook.ipynb

MauroLuzzatto/Q-Learning-Demo-Play

This repository contains a Jupyter Notebook implementation of a Q-Learning Agent, which learns to solve the n-Chain OpenAI...

1 Contributor 0 Issues 1 Star

Google Colaboratory

https://colab.research.google.com/github/Hvass-Labs/TensorFlow-Tutorials/blob/master/16_Reinforcement_Learning.ipynb#scrollTo=liUjEapzBhWp



Google Colaboratory

https://colab.research.google.com/github/Hvass-Labs/TensorFlow-Tutorials/blob/master/16_Reinforcement_Learning.ipynb



RL is very sample inefficient → needs a lot of time for training to be equivalent to a human trader.
Reward function is hard to design → you can do whatever you want.

Deep Q- Learning: from one state to another, you have probabilities.



Hyperparameters: